Inteligencia Artificial

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

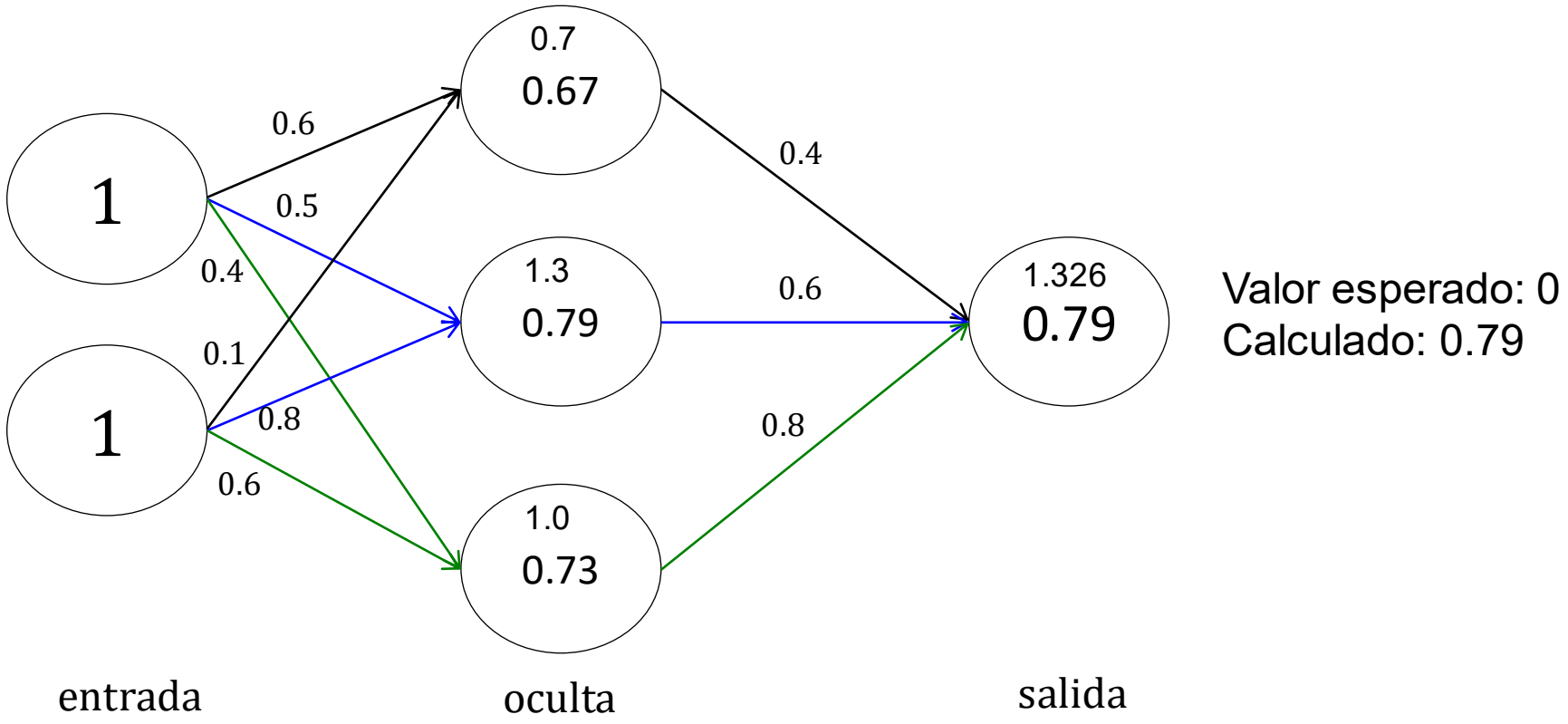
Facultad de Ingeniería en Electricidad y Computación – FIEC

Clase # 10 - PAO 2026 – I

Agenda

1. Aprendizaje en las redes MLP
   1.1 Forward propagation – Otro ejemplo
   1.2 Corrección del error – Back propagation
2. Algoritmo gradiente descendiente en las redes MLP
3. Las formas de entrada, salida
   3.1 Tensores

Propagación hacia delante - forward propagation



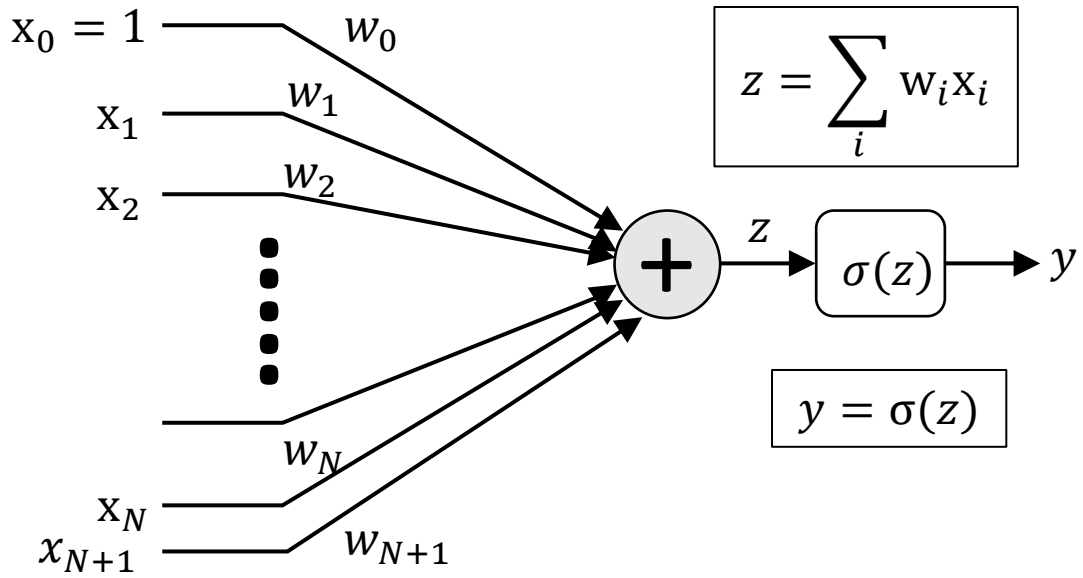
Valor esperado: 0
Calculado: 0.79

Propagación hacia delante - forward propagation

- ✓ Debido a que usamos un conjunto aleatorio de pesos iniciales, el valor total en la neurona de salida está errado: +0.79 (y el objetivo era 0)
- ✓ Si detenemos aquí el aprendizaje, este conjunto de pesos no servirá para reconocer los patrones de la entrada (operación XOR)
- ✓ Cómo calcular el error entre el valor esperado (0) y la salida real de la red (+0.79)?
- ✓ Luego de calcular el error, podemos aplicar la regla de aprendizaje Delta para aprender (corregir / descubrir) los pesos de la red, utilizando:

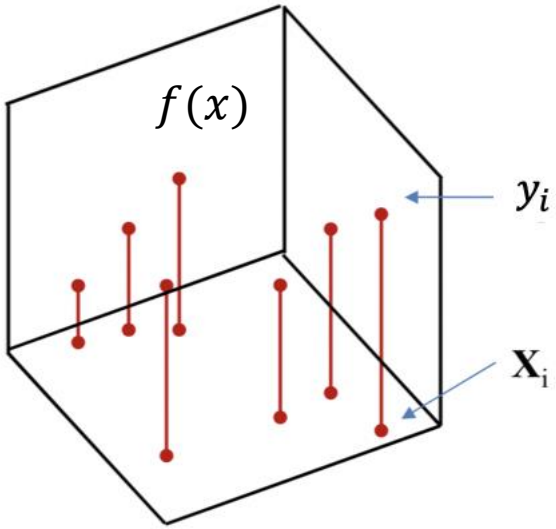
$$w_{nuevo} = w_{anterior} + \alpha(t - y)x$$

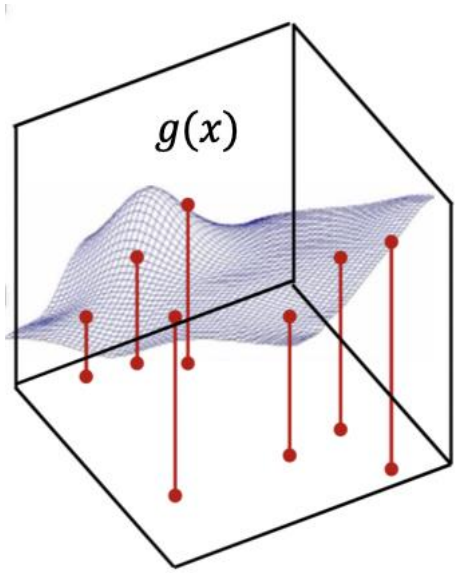
Aprendizaje en red



- El bias también se puede ver como el peso de otro componente de entrada que está fijo en 1
- Notar que: Si el bias no se menciona de manera explícita, siempre está. Recuerde que cada perceptron siempre tiene una entrada adicional (bias) que está fija en 1

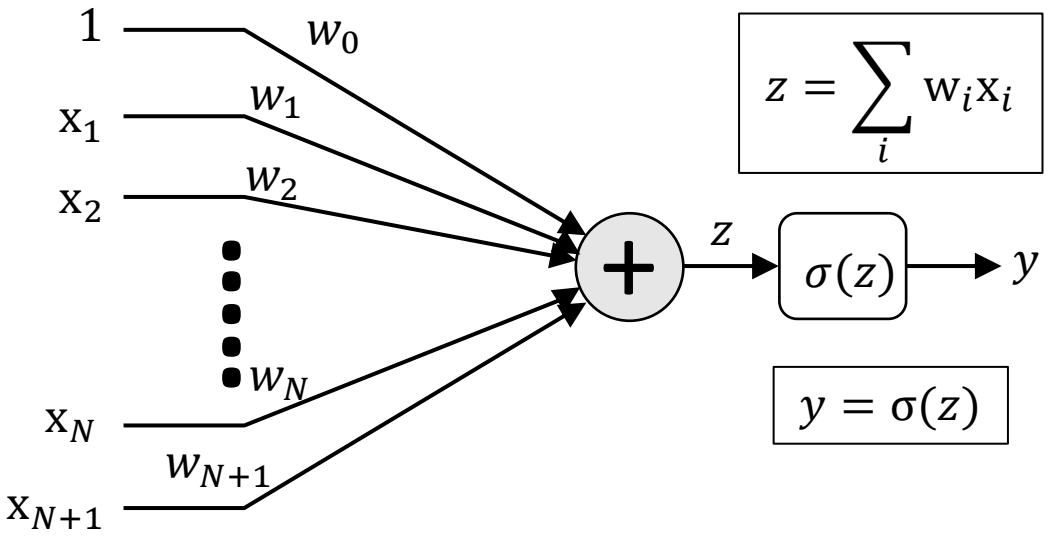
Aprendizaje de la función $g(X)$





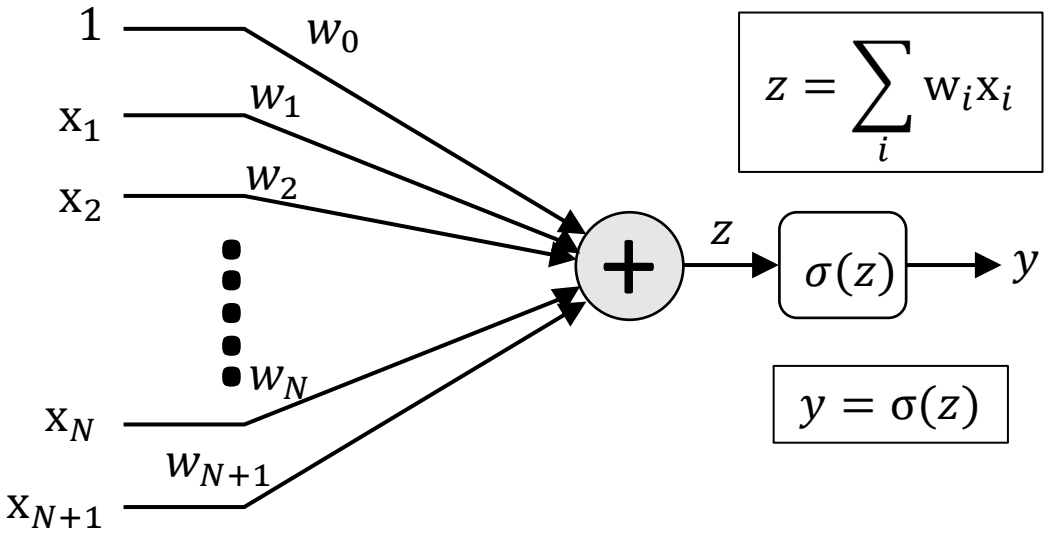
- Buscamos aprender la función $g(X_i)$ completamente a partir de pocos ejemplos….los ejemplos de entrenamiento

Cómo aprendemos la función $g(X)$?



- y es una función que depende de z; $\sigma(Z)$
- z es una función que depende de w_i y x_i
- Esto quiere decir que podemos calcular los cambios en la salida y, cuando ocurren pequeños cambios en la entrada x o en los pesos w
- El mecanismo que nos permite determinar esos cambios, de manera iterativa, es el gradiente descendiente !

Aprendizaje: Perceptron con activación derivable

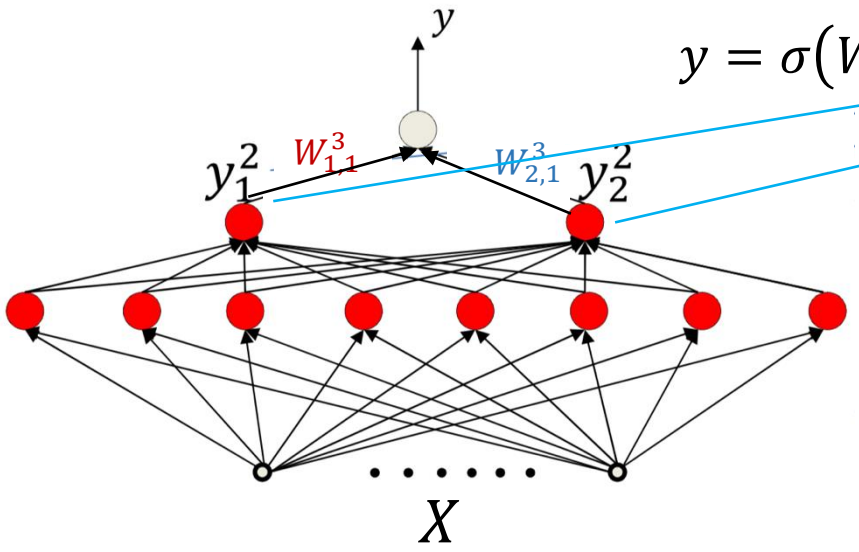


$$\frac{dy}{dw_i} = \frac{dy}{dz}\frac{dz}{dw_i}$$

$$\frac{dy}{dX_i} = \frac{dy}{dz}\frac{dz}{dX_i}$$

- Estas derivadas nos dicen cuan sensible es una función con respecto a las variables que la definen; es decir, cómo un pequeño cambio de las variables impacta al valor total de la función
- Durante el aprendizaje queremos ajustar los pesos, cuando se producen pequeños cambios en la función que define el error total

Aprendizaje: Red MLP derivable



$$y = \sigma\left(W_{1,1}^3 y_1^2 + W_{2,1}^3 y_2^2\right)$$

y = salida total de la red

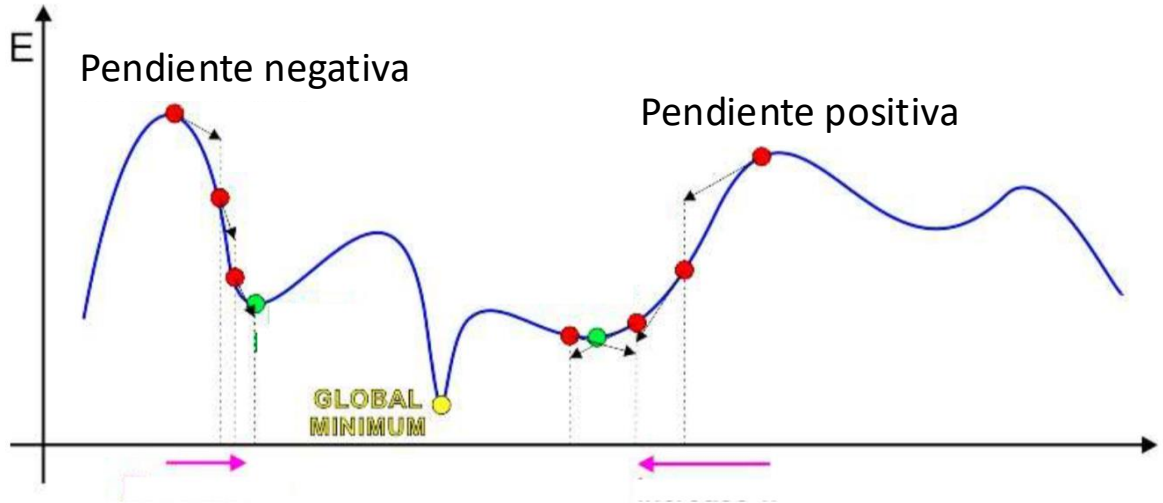
$W_{i,j}^k$ = peso de la conexión de la unidad i^{th} de la capa k^{th} a la unidad j^{th} de la capa $(k+1)^{th}$

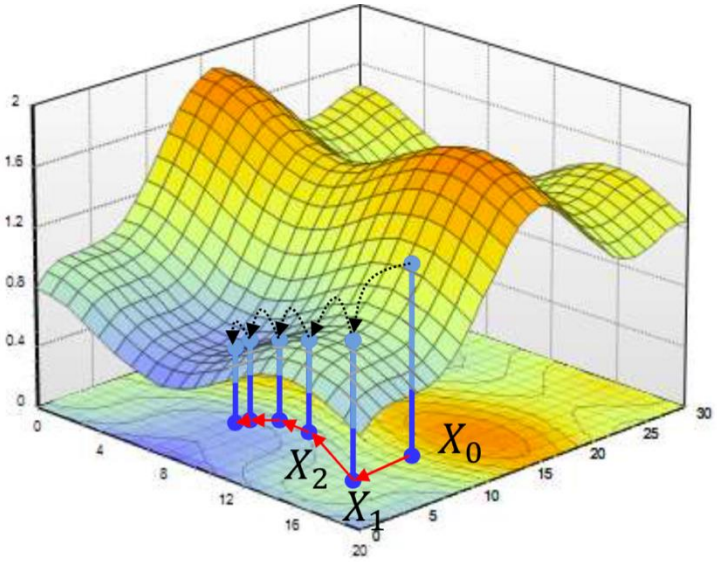
y_i^k = salida de la unidad i^{th} de la capa k^{th}

$\sigma(\,)$ = función de activación o umbral (derivable c.r.a w, x)

- Cada perceptron es derivable c.r.a sus pesos (incluyendo el peso del "bias") y "entradas"
- En Backprop se calcula la derivada de la función que define el error con respecto a cada peso w; así, determinamos cuán sensible es la función de error con respecto a cada uno de estos pesos
- Esto nos ayuda a determinar cuánto debemos cambiar cada parámetro w durante el descenso del gradiente; al final obtenemos el modelo que mejor se ajusta a los datos de entrenamiento

Aprendizaje: Gradiente descendiente

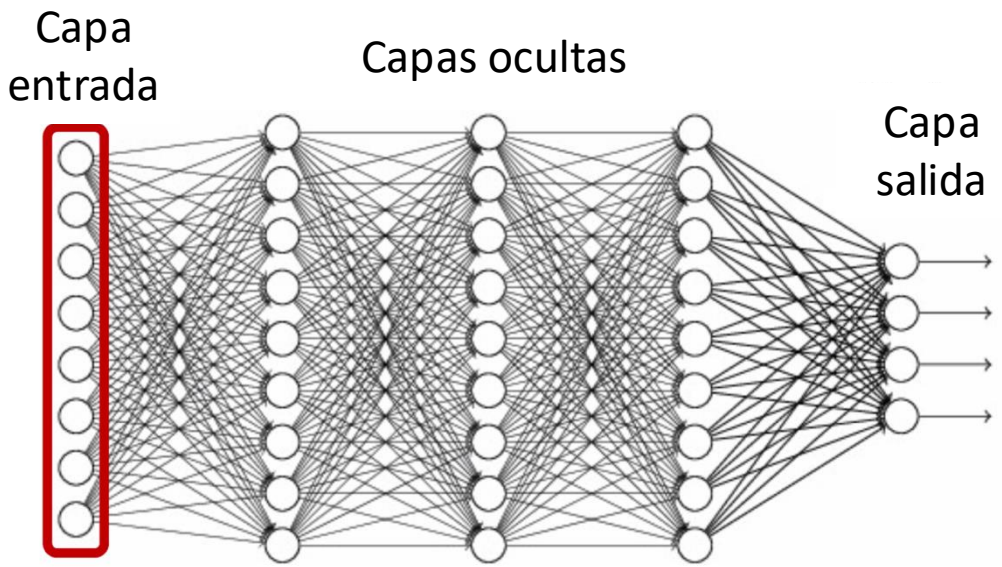




Esto es gradiente descendiente

- Solución iterativa
  - Empezar en un punto inicial (aleatorio)
  - Encontrar el punto en que cambia la dirección, para disminuir el error
    - Esto es, encontrar la derivada de la función:
      - Una derivada positiva, moverse a la izquierda disminuye el error
      - Una derivada negativa, moverse a la derecha disminuye el error
  - Cambiar la dirección en ese punto

Aprendizaje: La entrada



- Vector de números
  - Inclusive un valor escalar; tamaño del vector de entrada 1
  - Vector con valores de pixeles
  - Vector con valores de voz
  - Vector con valores reales representando texto
  - Vector con valores representando la salida de sensores, Otros..

Aprendizaje: La entrada - Tensores

Escalar: 3 Vector: $\begin{bmatrix}1\\2\\3\end{bmatrix}$ Matriz: $\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix}$ Tensor: $\begin{bmatrix}[1 & 2] & [3 & 4]\\[5 & 6] & [7 & 8]\end{bmatrix}$

- Tensores
  - Escalar: Tensor de dimensión 0 (0D). Por lo tanto, tiene 0 ejes y es de rango 0
  - Vector: Tensor de una sola dimensión (1D). Un vector tiene 1 eje y es de rango 1
  - Matriz: Tensor de rango 2, o 2 ejes, organizada en filas y columnas; técnicamente es un Tensor de 2 dimensiones (2D)
  - Tensor 3D (y mayor dimensionalidad): Si bien, técnicamente, todas las estructuras anteriores son tensores válidos, los tensores generalmente se refieren a las matrices de 3 o más dimensiones